

Mamba Documentation

The Mamba Programming Language

Simple syntax. Native performance. Multiple targets.

1. Introduction

1.1 What is Mamba?

Mamba is a multi-target programming language designed to combine readable syntax with native compilation and built-in web development capabilities.

1.2 Why Mamba?

```
Python-like readability
+
Native C++ performance
+
Built-in Web primitives
+
Built-in tooling
↓
MAMBA
```

1.3 Design Philosophy

- One source → multiple targets
- Simple syntax
- Native execution
- Web development as a first-class capability
- Minimal deployment footprint

1.4 Architecture

```
Mamba Source
|
▼
Compiler
/ \
▼  ▼
C++20 PHP
|
clang++
|
▼
Native Binary
```

1.5 Installation

Requirements: macOS/Linux, Python, clang++, SQLite.

Clone the repository and initialize the compiler:

```
git clone https://github.com/Muaviatanveer/Mamba.git
cd Mamba
python3 -m venv venv
source venv/bin/activate
pip install lark
chmod +x mamba
```

Verify the installation:

```
./mamba --help
```

1.6 Hello World

Write your first Mamba program. Create hello.mb:

```
print("Hello, Mamba!")
```

Run it:

```
./mamba hello.mb
```

Output:

```
Hello, Mamba!
```

2. Language Basics

2.1 Comments

```
# This is a comment
// This is also a comment
```

2.2 Variables

Variables are declared using the explicit let keyword. They can hold various data types.

```
let name = "Muavia"
let age = 20
let active = true

print(name)
print(age)
```

2.3 Data Types

String:

```
let name = "Alex"
```

Number:

```
let age = 25  
let price = 99.5
```

Boolean:

```
let active = true
```

Array:

```
let fruits = ["Apple", "Banana", "Cherry"]
```

HashMap:

```
let user = {  
  "name": "Alex",  
  "age": 25  
}
```

2.4 Arrays

You can manipulate arrays dynamically using standard functions.

```
let fruits = ["Apple", "Banana"]  
arr.push(fruits, "Cherry")  
arr.pop(fruits)
```

2.5 HashMaps

Dictionaries support dynamic key access.

```
let user = { "name": "Muavia" }  
print(user["name"])
```

2.6 Functions

Functions are declared with fn.

```
fn add(a, b) {  
  return a + b  
}  
  
let result = add(10, 20)  
print(result)
```

2.7 Conditions

```
if age >= 18 {  
  print("Adult")  
} else {  
  print("Minor")  
}
```

2.8 Loops

```
let x = 0  
while x < 10 {  
  x = x + 1  
}
```

2.9 Error Handling

```
try {  
  let data = file.read("data.txt")  
} catch (err) {  
  print(err)  
}
```

2.10 Imports

Import other .mb files easily.

```
import "helpers.mb"
```

3. Standard Library

print

Outputs data to stdout.

```
print("Log message")
```

env

Access system environment variables.

```
let port = env.get("PORT")
```

json

Parse and stringify JSON objects.

```
let user = { "name": "Alex", "age": 25 }  
let payload = json.stringify(user)  
let parsed = json.parse(payload)
```

file

Read and write text files.

```
file.write("hello.txt", "Hello Mamba")
let content = file.read("hello.txt")
```

http

Send native HTTP client requests.

```
let response = http.get("https://example.com")
```

db

Open embedded SQLite databases natively.

```
let conn = db.open("app.db")
```

arr

Array manipulations.

```
arr.push(fruits, "Orange")
arr.contains(fruits, "Banana")
arr.join(fruits, " | ")
```

map

HashMap manipulations.

```
map.has(user, "name")
map.remove(user, "temp_key")
```

4. Web Development

Web development in Mamba does not require external frameworks. Endpoints are first-class language constructs.

Routes & Response

```
route GET "/api/hello" {
  return json.stringify({
    "message": "Hello from Mamba"
  })
}
```

Request Context

Mamba injects a req context variable inside routes.

Query parameters:

```
let search = req.query("search")
```

Headers:

```
let auth = req.header("Authorization")
```

Request body:

```
let body = req.body()
let payload = json.parse(body)
```

CORS

OPTIONS requests are automatically managed, and Mamba routes automatically return Access-Control-Allow-Origin: * headers natively.

5. Database

Mamba seamlessly binds SQLite, allowing zero-dependency database access.

Parameterized Queries (Safe SQL)

```
let db_conn = db.open("app.db")

db.query(
  "INSERT INTO users (name, role) VALUES (?, ?)",
  ["Muavia", "developer"]
)
```

Reading Results

```
let users = db.query("SELECT * FROM users")
```

6. CLI Tooling

- mamba init <app>: Scaffolds a new project.
 - mamba build <file.mb> --release: Compiles optimized binary.
 - mamba test <file.mb>: Runs native test assertions.
 - mamba fmt <file.mb>: Formats source code.
 - mamba deploy: Deploys to Mamba Cloud Runtime.
 - mamba proxy <port>: Starts the reverse proxy gateway.
 - mamba git-init <app>: Sets up push-to-deploy git server.
-

7. Mamba Cloud Deployment

Mamba shifts from a language to an ecosystem by offering its own built-in PaaS engine.

```
# Build the native binary
./mamba build app.mb --release

# Run Proxy Gateway on port 8000
./mamba proxy 8000

# Deploy instantly
./mamba deploy 8000

# Make it public globally via edge bridge
./mamba deploy 8000 --public
```

8. Build a Production Application with Mamba

This tutorial takes you from zero to a globally deployed REST API.

1. Scaffold your project

```
./mamba init my-app
cd my-app
```

2. Write main.mb

```
let conn = db.open("app.db")
db.query("CREATE TABLE IF NOT EXISTS items (id INTEGER PRIMARY KEY, name TEXT)")

route POST "/api/items" {
  let payload = json.parse(req.body())
  db.query("INSERT INTO items (name) VALUES (?)", [payload["name"]])

  return json.stringify({ "status": "created", "item": payload["name"] })
}
```

3. Setup Push-to-Deploy Server

```
./mamba git-init my_app
git remote add mamba_cloud ../mamba_cloud_repos/my_app.git
```

4. Deploy to the world

```
git push mamba_cloud main
```

5. Visit your URL!

<https://my-app.mamba.cloud> (or via Cloudflare Edge Bridge!)

9. Performance & Benchmarks

Hardware: macOS 15.5, Apple Clang 17, M1/M2 Max.

Methodology: ApacheBench 50,000 requests, 1,000 concurrency.

Language	Throughput	Failures
----------	------------	----------

Mamba C++ Native (-O3)	19,865 req/s 0
Rust (rustc -O)	22,993 req/s 0
PHP (cli-server)	15,806 req/s 0
Python (http.server)	15,358 req/s 0

10. Architecture

Compiler: Preprocessor → Parser (Lark LALR) → C++20 Target Engine → Clang++ (-O3)

Runtime Web Server: Embedded POSIX sockets with a scalable native std::thread worker pool handling concurrent JSON routing and SQLite std::mutex executions securely.

Deployment Engine: Background Daemon process manager parsing deployments.json and a Header-based Domain Router.

11. Contributing

1. Fork the repository
2. Branch off main for your feature
3. Implement changes
4. Test using ./mamba test
5. Submit a PR